

Lab 03: Verbos principales para manipulacion de datos

Actividad

Fase 1: Aprendizaje independiente en su grupo original (3 minutos)

- Nos dividimos en grupos de 5 personas.
- Cada grupo tiene un experto para los distintos aspectos:
 1. características de los datos tidy y la manipulación con pipelines
 2. operaciones a nivel de **tabela**
 3. operaciones a nivel de **fila**
 4. operaciones a nivel de **columna**
 5. uniendo datos
- El experto, estudia el material sobre su enfoque específico.

Fase 2: Aprendizaje colaborativo en grupos de expertos (3 minutos)

- Los expertos de cada enfoque se reúnen en grupos para compartir sus conocimientos.
- Aclaran dudas entre expertos, toman notas y se preparan para enseñar a su grupo original.
- El experto prepara un reporte breve de:
 1. ¿Qué hace esta operación?
 2. ¿Qué cambia: las filas, las columnas, los valores, o el número de observaciones?
 3. Dé un ejemplo ecológico de cuándo la usaría. De la experiencia reciente con el analisis de datos en su grupo, mencione un caso concreto.

Fase 3: Enseñanza en el grupo original (5 minutos)

- Cada experto regresa a su grupo original y enseña a los demás miembros del grupo sobre su área específica.
- Se discuten y resuelven dudas sobre las temas presentados por los expertos.

Fase 4: Reflexión y cierre (5 minutos)

- Se comparten conclusiones y se resuelven dudas finales con toda la clase.
- Cada grupo original (no grupo de expertos) tiene que resolver la situación de la siguiente página (caja roja).

! Situación (Fase 4)

Cada grupo original (no grupo de expertos) tiene que resolver la siguiente situación:

Tenemos una tabla llamada *muestras* con la abundancia (conteo) de macroinvertebrados de 4 sitios, medida en varias fechas; cada muestra cubre un área fija de 0.25 m².

sitio	fecha	conteo
-------	-------	--------

Tenemos una segunda tabla llamada *sitios*, con un solo valor de temperatura y oxígeno por cada uno de los 4 sitios (estos valores no varían por fecha).

sitio	temperatura	oxígeno
-------	-------------	---------

Queremos, a partir de la tabla *muestras*:

- quedarnos solamente con las muestras de septiembre (columna fecha);
- agregar una columna de densidad (individuos/m²), dividiendo el conteo entre el área de la muestra;
- calcular la densidad promedio de septiembre por sitio;
- ordenar los sitios de mayor a menor densidad promedio;
- agregar, para cada sitio, la temperatura y oxígeno desde la tabla *sitios*.

¿Qué operaciones necesitamos y en qué orden? ¿Con qué columna(s) se debe hacer la unión (join) entre las dos tablas?

Material

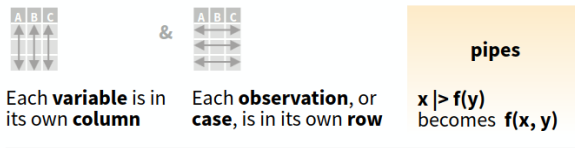
En lo que sigue se presentan las principales funciones de manipulación de datos en R utilizando el enfoque tidy data. Todos los imágenes provienen del dplyr cheatsheet disponible en el siguiente enlace: [dplyr cheatsheet](#).

Tidy data (Datos ordenados) (experto 1)

Datos tidy:

- Cada columna representa una variable.
- Cada fila representa una observación.

dplyr functions work with pipes and expect **tidy data**. In tidy data:



Pipelines, el operador `|>` o `%>%`:

- Facilitan la manipulación de datos tidy.
- Toman la salida de una operación y la pasan como entrada a la siguiente operación
- Esto permite encadenar múltiples transformaciones de manera clara y concisa.

```
datos |> filter(variable > valor) |> summarise(promedio = mean(variable))
```

Operaciones a nivel de tabela (experto 2)

Resúmenes

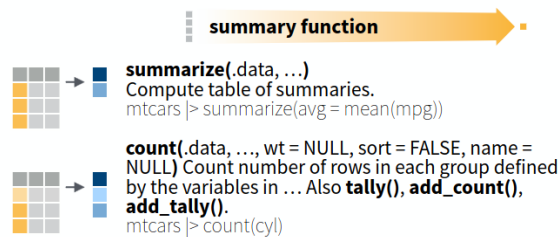
`summarise()`:

- Aplica una función para generar nueva tabela con resúmenes.
- Permite obtener estadísticas descriptivas.
- Se utiliza comúnmente junto con `group_by()` para obtener resúmenes por grupo.

```
datos |> summarise(promedio = mean(variable))
```

Summarize Cases

Apply **summary functions** to columns to create a new table of summary statistics. Summary functions take vectors as input and return one value (see back).



Agrupamiento

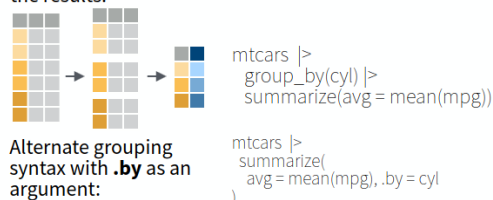
`group_by()`:

- Agrupa los datos según una o más variables.
- Crea version modificada de los datos con el agrupamiento aplicado.
- Todas las operaciones posteriores se aplican dentro de cada grupo.
- Se utiliza comúnmente junto con `summarise()` para obtener resúmenes por grupo.

```
datos |> group_by(variable_categorica)
```

Group Cases

Use **group_by(.data, ..., .add = FALSE, .drop = TRUE)** to create a "grouped" copy of a table grouped by columns in ... dplyr functions will manipulate each "group" separately and combine the results.



Operaciones a nivel de fila (experto 3)

Filtrado (selección de filas)

`filter()`:

- Selecciona filas que cumplen con una condición específica.
- Permite filtrar los datos según valores de las columnas.
- Se utiliza comúnmente para enfocarse en subconjuntos relevantes de los datos.

`datos |> filter(variable > valor)`

Manipulate Cases

EXTRACT CASES

Row functions return a subset of rows as a new table.



filter(.data, ..., .preserve = FALSE) Extract rows that meet logical criteria.
`mtcars |> filter(mpg > 20)`

filter_out(.data, ..., .preserve = FALSE): Drop rows that meet logical criteria (the complement to `filter()`). Missing values in the dropped condition are kept by default, so you don't need to guard against NA explicitly.
`mtcars |> filter_out(mpg > 20)`

Use **when_any()** and **when_all()** inside **filter()** to combine multiple conditions.
`mtcars |> filter(when_any(mpg > 30, hp > 300))`

Ordenamiento de filas

`arrange()`:

- Ordena las filas de un data frame según una o más columnas.
- Permite especificar el orden ascendente o descendente.
- Facilita la visualización y el análisis de los datos en un orden específico.

`datos |> arrange(variable)`

ARRANGE CASES



arrange(.data, ..., .by_group = FALSE) Order rows by values of a column or columns (low to high), use with **desc()** to order from high to low.
`mtcars |> arrange(mpg)`
`mtcars |> arrange(desc(mpg))`

Operaciones a nivel de columna (experto 4)

Selección de columnas

`select()`:

- Permite elegir un subconjunto de columnas de un data frame.
- Facilita trabajar solo con las columnas relevantes.
- Se puede usar para reordenar columnas o renombrarlas.

```
datos |> select(columna1, columna2)
```

Manipulate Variables



EXTRACT VARIABLES

Column functions return a set of columns as a new vector or table.



`pull(.data, var = -1, name = NULL, ...)` Extract column values as a vector, by name or index.
`mtcars |> pull(wt)`



`select(.data, ...)` Extract columns as a table.
`mtcars |> select(mpg, wt)`



`relocate(.data, ..., .before = NULL, .after = NULL)` Move columns to new position.
`mtcars |> relocate(mpg, cyl, .after = last_col())`

Creando nuevas columnas

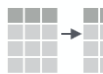
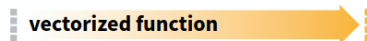
`mutate()`:

- Crea nuevas columnas o modifica las existentes.
- Permite realizar cálculos basados en otras columnas.
- Se utiliza comúnmente para agregar información derivada a los datos.

```
datos |> mutate(nueva_columna = variable1 + variable2)
```

MAKE NEW VARIABLES

Apply **vectorized functions** to columns. Vectorized functions take vectors as input and return vectors of the same length as output (see back).



`mutate(.data, ..., .keep = "all", .before = NULL, .after = NULL)` Compute new column(s). Also **`add_column()`**.
`mtcars |> mutate(gpm = 1 / mpg)`
`mtcars |> mutate(gpm = 1 / mpg, .keep = "none")`



`rename(.data, ...)` Rename columns. Use **`rename_with()`** to rename with a function.
`mtcars |> rename(miles_per_gallon = mpg)`

Uniando datos (experto 5)

Uniando datos sin claves

`bind_rows()`:

- Permite combinar varias tablas con las mismas columnas.

`bind_rows(tabla1, tabla2)`

Combine Tables

COMBINE VARIABLES

x + y =

A	B	C
a	t	1
b	u	2
c	v	3

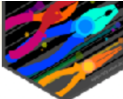
bind_cols(..., name_repair) Returns tables placed side by side as a single table. Column lengths must be equal. Columns will NOT be matched by id (to do that look at Relational Data below), so be sure to check that both tables are ordered the way you want before binding.

COMBINE CASES

x + y =

A	B	C
a	t	1
b	u	2
c	v	3

bind_rows(..., id=NULL) Returns tables one on top of the other as a single table. Set .id to a column name to add a column of the original table names (as pictured).



Uniando datos con claves

`left_join()`:

- Permite combinar dos tablas según una clave común, conservando todas las filas de la tabla izquierda.
- Se utiliza comúnmente para agregar información adicional a un conjunto de datos existente.
- No conserva las filas de la tabla derecha que no tienen coincidencias en la tabla izquierda.

`left_join(tabla1, tabla2, by = "clave")`

RELATIONAL DATA

x + y =

A	B	C
a	t	1
b	u	2
c	v	3

RELATIONAL DATA

y

A	B	D
a	t	3
b	u	2
d	w	1

Use a **"Mutating Join"** to join one table to columns from another, matching values with the rows that they correspond to. Each join retains a different combination of values from the tables.

left_join(x, y, by = NULL, copy = FALSE, suffix = c(".x", ".y"), ..., keep = FALSE, na_matches = "na") Join matching values from y to x.

A	B	C	D
a	t	1	3
b	u	2	2
c	v	3	NA

Hay otros tipos de uniones con claves, como `right_join()`, `inner_join()` y `full_join()`, que permiten combinar tablas de diferentes maneras según las necesidades del análisis.